

12.6 Fourier Transforms of Real Data in Two and Three Dimensions

Two-dimensional FFTs are particularly important in the field of image processing. An image is usually represented as a two-dimensional array of pixel intensities, real (and usually positive) numbers. One commonly desires to filter high, or low, frequency spatial components from an image; or to convolve or deconvolve the image with some instrumental point spread function. Use of the FFT is by far the most efficient technique.

In three dimensions, a common use of the FFT is to solve Poisson's equation for a potential (e.g., electromagnetic or gravitational) on a three-dimensional lattice that represents the discretization of three-dimensional space. Here the source terms (mass or charge distribution) and the desired potentials are also real. In two and three dimensions, with large arrays, memory is often at a premium. It is therefore important to perform the FFTs, insofar as possible, on the data "in place." We want a routine with functionality similar to the multidimensional FFT routine `fourn` (§12.5), but which operates on real, not complex, input data. We give such a routine in this section. The development is analogous to that of §12.3 leading to the one-dimensional routine `realfft`. (You might wish to review that material at this point, particularly equation 12.3.6.)

It is convenient to think of the independent variables $n_1, \dots, n_L$ in equation (12.5.3) as representing an L -dimensional vector $\vec{n}$ in wave-number space, with values on the lattice of integers. The transform $H(n_1, \dots, n_L)$ is then denoted $H(\vec{n})$.

It is easy to see that the transform $H(\vec{n})$ is periodic in each of its L dimensions. Specifically, if $\vec{P}_1, \vec{P}_2, \vec{P}_3, \dots$ denote the vectors $(N_1, 0, 0, \dots)$, $(0, N_2, 0, \dots)$, $(0, 0, N_3, \dots)$, and so forth, then

$$H(\vec{n} \pm \vec{P}_j) = H(\vec{n}) \quad j = 1, \dots, L \quad (12.6.1)$$

Equation (12.6.1) holds for any input data, real or complex. When the data are real, we have the additional symmetry

$$H(-\vec{n}) = H(\vec{n})^* \quad (12.6.2)$$

Equations (12.6.1) and (12.6.2) imply that the full transform can be trivially obtained from the subset of lattice values $\vec{n}$ that have

$$\begin{aligned} 0 &\leq n_1 \leq N_1 - 1 \\ 0 &\leq n_2 \leq N_2 - 1 \\ &\dots \\ 0 &\leq n_L \leq \frac{N_L}{2} \end{aligned} \quad (12.6.3)$$

In fact, this set of values is overcomplete, because there are additional symmetry relations among the transform values that have $n_L = 0$ and $n_L = N_L/2$. However, these symmetries are complicated and their use becomes extremely confusing. Therefore, we will compute our FFT on the lattice subset of equation (12.6.3), even though this requires a small amount of extra storage for the answer, i.e., the transform

is not *quite* “in place.” (Although an in-place transform is in fact possible, we have found it virtually impossible to explain to any user how to unscramble its output, i.e., where to find the real and imaginary components of the transform at some particular frequency!)

As a “bare” routine, we will implement the multidimensional real Fourier transform for the three-dimensional case $L = 3$, with the input data stored as a real three-dimensional array `data[0..nn1-1][0..nn2-1][0..nn3-1]`. This scheme will allow two-dimensional data to be processed with effectively no loss of efficiency simply by choosing $nn1 = 1$. (Note that it must be the *first* dimension that is set to 1.) We also provide more convenient overloaded functions whose input data are stored as a `Mat3DDoub` (for three-dimensional data) or as a `MatDoub` (for two-dimensional data).

The output spectrum comes back packaged, logically at least, as a *complex* three-dimensional array that we can call `spec[0..nn1-1][0..nn2-1][0..nn3/2]` (cf. equation 12.6.3). In the first two of its three dimensions, the respective frequency values f_1 or f_2 are stored in wraparound order, that is, with zero frequency in the first index value, the smallest positive frequency in the second index value, the smallest *negative* frequency in the *last* index value, and so on (cf. the discussion leading up to routines `four1` and `fourn`). The third of the three dimensions returns only the positive half of the frequency spectrum. Figure 12.6.1 shows the logical storage scheme. The returned portion of the complex output spectrum is shown as the unshaded part of the lower figure.

The physical, as opposed to logical, packaging of the output spectrum is necessarily a bit different from the logical packaging, because, counting components, `spec` doesn’t quite fit into `data`. The subscript range `spec[0..nn1-1][0..nn2-1][0..nn3/2-1]` is returned in the input array `data[0..nn1-1][0..nn2-1][0..nn3-1]`, with the correspondence

$$\begin{aligned} \text{Re}(\text{spec}[i1][i2][i3]) &= \text{data}[i1][i2][2*i3] \\ \text{Im}(\text{spec}[i1][i2][i3]) &= \text{data}[i1][i2][2*i3+1] \end{aligned} \quad (12.6.4)$$

The remaining “plane” of values, `spec[0..nn1-1][0..nn2-1][nn3/2]`, is returned in the two-dimensional `MatDoub` array `speq[0..nn1-1][0..2*nn2-1]`, with the correspondence

$$\begin{aligned} \text{Re}(\text{spec}[i1][i2][nn3/2]) &= \text{speq}[i1][2*i2] \\ \text{Im}(\text{spec}[i1][i2][nn3/2]) &= \text{speq}[i1][2*i2+1] \end{aligned} \quad (12.6.5)$$

Note that `speq` contains only frequency components whose third component f_3 is at the Nyquist critical frequency $\pm f_c$. In some applications these values will in fact be ignored or set to zero, since they are intrinsically aliased between positive and negative frequencies.

With this much introduction, the implementing procedure, called `rlft3`, is something of an anticlimax. Look in the innermost loop in the procedure, and you will see equation (12.3.6) implemented on the *last* transform index. The case of $i3=0$ is coded separately, to account for the fact that `speq` is to be filled instead of overwriting the input array of `data`. The three enclosing `for` loops (indices $i2$, $i3$, and $i1$, from inside to outside) could in fact be done in any order — their actions all commute. We chose the order shown because of the following considerations: (i)

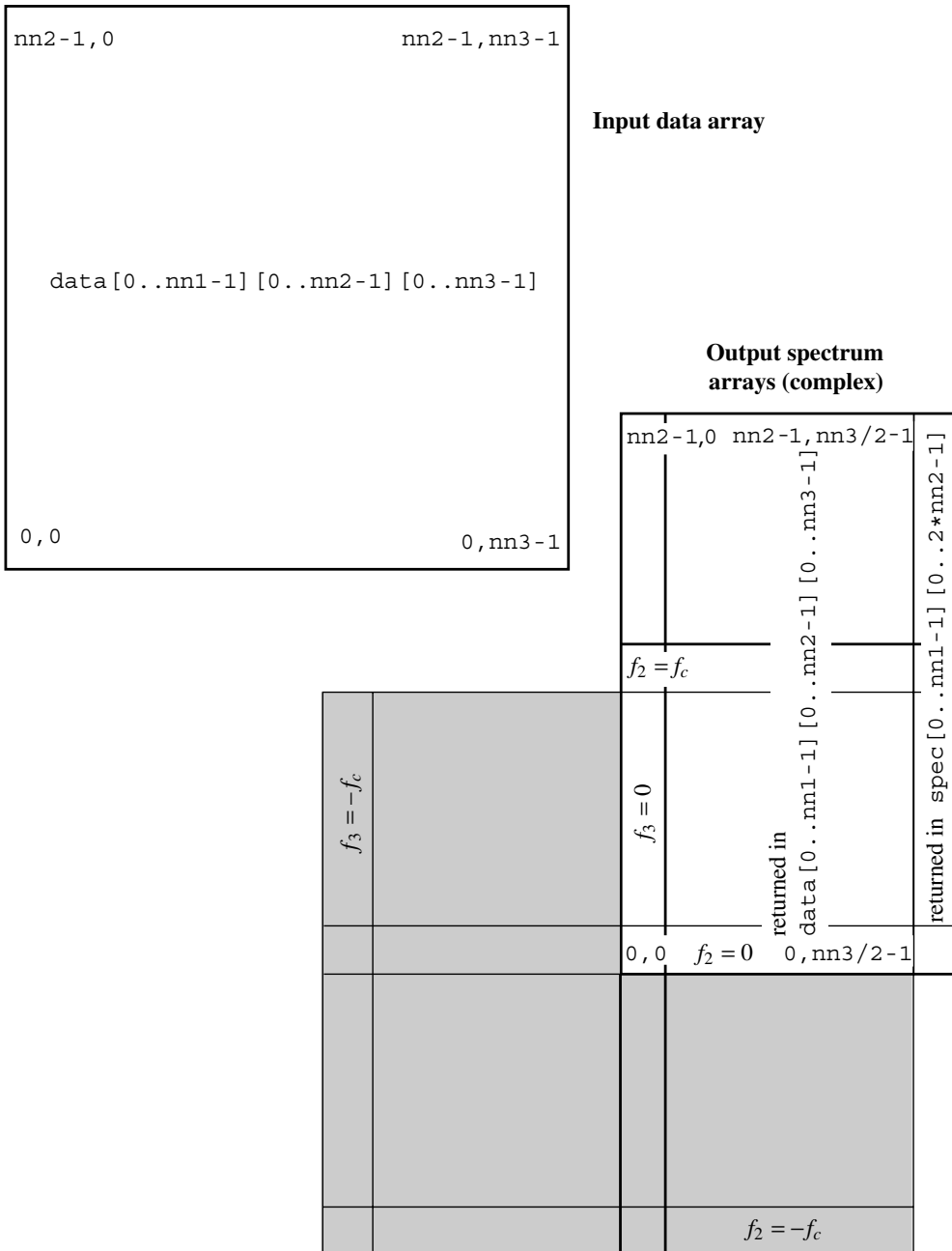


Figure 12.6.1. Input and output data arrangements for `rlft3`. All arrays shown are presumed to have a first (leftmost) dimension of range $[0..nn1-1]$, coming out of the page. The input data array is a real three-dimensional array `data[0..nn1-1][0..nn2-1][0..nn3-1]`. (For two-dimensional data, one sets $nn1 = 1$.) The output data can be viewed as a single complex array with dimensions $[0..nn1-1][0..nn2-1][0..nn3/2]$ (cf. equation 12.6.3), corresponding to the frequency components f_1 and f_2 being stored in wraparound order, but only positive f_3 values being stored (others being obtainable by symmetry). The output data are actually returned mostly in the input array `data`, but partly stored in the real array `spec[0..nn1-1][0..2*nn2-1]`. See text for details.

i3 should not be the inner loop, because if it is, then the recurrence relations on `wr` and `wi` become burdensome. (ii) On modern processors with a cache hierarchy, `i1` should be the outer loop, because (with C++ order of array storage) this results in the array data, which might be very large, being accessed in block sequential order.

Keep in mind that all the computing in `rlft3` is negligible, by a logarithmic factor, compared with the actual work of computing the associated complex FFT, done in the routine `fourn`. Complex operations are carried out explicitly in terms of real and imaginary parts. The routine `rlft3` is based on an earlier routine by G.B. Rybicki. As previously, it is convenient to provide a bare routine, and then more convenient overloaded functions. The overload for three-dimensional data inputs the data as a `Mat3DDoub`, with `speq` a `MatDoub`. The overload for two-dimensional data inputs the data as a `MatDoub`, with `speq` a `VecDoub`.

fourier_ndim.h

```
void rlft3(Doub *data, Doub *speq, const Int isign,
           const Int nn1, const Int nn2, const Int nn3) {
    Given a three-dimensional real array data[0..nn1-1][0..nn2-1][0..nn3-1] (where nn1 = 1
    for the case of a logically two-dimensional array), this routine returns (for isign=1) the complex
    fast Fourier transform as two complex arrays: On output, data contains the zero and positive
    frequency values of the third frequency component, while speq[0..nn1-1][0..2*nn2-1] con-
    tains the Nyquist critical frequency values of the third frequency component. First (and second)
    frequency components are stored for zero, positive, and negative frequencies, in standard
    wraparound order. See text for description of how complex values are arranged. For isign=-1,
    the inverse transform (times nn1*nn2*nn3/2 as a constant multiplicative factor) is performed,
    with output data (viewed as a real array) deriving from input data (viewed as complex) and
    speq. For inverse transforms on data not generated first by a forward transform, make sure the
    complex input data array satisfies property (12.6.2). The dimensions nn1, nn2, nn3 must always
    be integer powers of 2.

    Int i1,i2,i3,j1,j2,j3,k1,k2,k3,k4;
    Doub theta,wi,wpi,wpr,wr,wtemp;
    Doub c1,c2,h1r,h1i,h2r,h2i;
    VecInt nn(3);
    VecDoub spq(nn1);
    for (i1=0;i1<nn1;i1++) spq[i1] = speq + 2*nn2*i1;
    c1 = 0.5;
    c2 = -0.5*isign;
    theta = isign*(6.28318530717959/nn3);
    wtemp = sin(0.5*theta);
    wpr = -2.0*wtemp*wtemp;
    wpi = sin(theta);
    nn[0] = nn1;
    nn[1] = nn2;
    nn[2] = nn3 >> 1;
    if (isign == 1) {
        Case of forward transform.
        Here is where most all of the com-
        pute time is spent.
        Extend data periodically into speq.
        furn(data,nn,isign);
        k1=0;
        for (i1=0;i1<nn1;i1++)
            for (i2=0,j2=0;i2<nn2;i2++,k1+=nn3) {
                spq[i1][j2++] = data[k1];
                spq[i1][j2++] = data[k1+1];
            }
    }
    for (i1=0;i1<nn1;i1++) {
        j1=(i1 != 0 ? nn1-i1 : 0);
        Zero frequency is its own reflection, otherwise locate corresponding negative frequency
        in wraparound order.
        wr=1.0;
        wi=0.0;
        for (i3=0;i3<=(nn3>>1);i3+=2) {
            k1=i1*nn2*nn3;
            k3=j1*nn2*nn3;
```

```

    for (i2=0; i2<nn2; i2++, k1+=nn3) {
        if (i3 == 0) {
            Equation (12.3.6).
            j2=(i2 != 0 ? ((nn2-i2)<<1) : 0);
            h1r=c1*(data[k1]+spq[j1][j2]);
            h1i=c1*(data[k1+1]-spq[j1][j2+1]);
            h2i=c2*(data[k1]-spq[j1][j2]);
            h2r=-c2*(data[k1+1]+spq[j1][j2+1]);
            data[k1]=h1r+h2r;
            data[k1+1]=h1i+h2i;
            spq[j1][j2]=h1r-h2r;
            spq[j1][j2+1]=h2i-h1i;
        } else {
            j2=(i2 != 0 ? nn2-i2 : 0);
            j3=nn3-i3;
            k2=k1+i3;
            k4=k3+j2*nn3+j3;
            h1r=c1*(data[k2]+data[k4]);
            h1i=c1*(data[k2+1]-data[k4+1]);
            h2i=c2*(data[k2]-data[k4]);
            h2r=-c2*(data[k2+1]+data[k4+1]);
            data[k2]=h1r+wr*h2r-wi*h2i;
            data[k2+1]=h1i+wr*h2i+wi*h2r;
            data[k4]=h1r-wr*h2r+wi*h2i;
            data[k4+1]=-h1i+wr*h2i+wi*h2r;
        }
        wr=(wtemp=wr)*wpr-wi*wpi+wr;
        wi=wi*wpr+wtemp*wpi+wi;
    }
    if (isign == -1) fourn(data, nn, isign);
}

void rlft3(Mat3DDoub_IO &data, MatDoub_IO &speq, const Int isign) {
    Overloaded version for three-dimensional data. When isign is 1, replace data and spec by
    data's three-dimensional FFT. When isign is -1, the inverse transform (times one-half the
    product of data's dimensions) is performed. See comments in version above.
    if (speq.nrows() != data.dim1() || speq.ncols() != 2*data.dim2())
        throw("bad dims in rlft3");
    rlft3(&data[0][0][0], &speq[0][0], isign, data.dim1(), data.dim2(), data.dim3());
}

void rlft3(MatDoub_IO &data, VecDoub_IO &speq, const Int isign) {
    Overloaded version for two-dimensional data. When isign is 1, replace data and spec by data's
    two-dimensional FFT. When isign is -1, the inverse transform (times one-half the product of
    data's dimensions) is performed. See comments in version above.
    if (speq.size() != 2*data.nrows()) throw("bad dims in rlft3");
    rlft3(&data[0][0], &speq[0], isign, 1, data.nrows(), data.ncols());
}

```

As in earlier sections of this chapter, we can use a bit of C++ trickery to access the output Fourier components (logical array `spec`) more easily. We define two overloaded helper functions (the first of which is identical to the definition in section §12.3)

```

inline Complex* Cmplx(VecDoub &d) {return (Complex *)&d[0];}
inline Complex* Cmplx(Doub *d) {return (Complex *)d;}

```

Now suppose that `data` is two-dimensional with input dimensions `nx` and `ny`. Then, on output, a complex frequency component (i, j) , with $0 \leq i \leq nx/2$ and $0 \leq j \leq ny/2 - 1$, can be accessed as

```
Cmplx(data[i])[j]
```



Figure 12.6.2. Fourier processing of an image. Upper left: Original image. Upper right: Blurred by low-pass filtering. Lower left: Sharpened by enhancing high frequency components. Lower right: Magnitude of the derivative operator as computed in Fourier space.

Yes, the right parenthesis really is between the subscripts! The corresponding negative (wraparound) frequencies are at

```
Cmplx(data[nx-i])[j]
```

but now with $1 \leq i \leq nx/2 - 1$. The Nyquist critical values $j = ny/2$ can be accessed as

```
Cmplx(speq)[i]
```

for $0 \leq i \leq nx/2$ and

```
Cmplx(speq)[nx-i]
```

for $1 \leq i \leq nx/2 - 1$. If you don't understand how this all works, a useful exercise is to locate each of these expressions in Figure 12.6.1. All of the above expressions can be l-values as well as r-values.

Figure 12.6.2 shows a test image* and three examples of processing with `r1ft3` (using the overloaded function for two-dimensional data). The first example is a simple low-pass filter. A sharp image becomes blurry when its high-frequency spatial

*We are inordinately fond of this 1950s vintage IEEE test image, despite the fact that many readers have urged us to use instead the historically important “Lenna” image from the early 1970s. See [1] for an interesting recounting of the history. “Lenna,” a strategically cropped Playboy centerfold, is also said to be the source of the term “discreet Fourier transform.”